

Weapon Class

1.0

Generated by Doxygen 1.16.1

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 Weapon Class Reference	5
3.1.1 Detailed Description	6
3.1.2 Constructor & Destructor Documentation	7
3.1.2.1 Weapon() [1/2]	7
3.1.2.2 Weapon() [2/2]	7
3.1.2.3 ~Weapon()	7
3.1.3 Member Function Documentation	7
3.1.3.1 computeCollisionRec()	7
3.1.3.2 Draw()	8
3.1.3.3 getAttackType()	8
3.1.3.4 getCollisionRect()	8
3.1.3.5 getDamage()	9
3.1.3.6 setAttackType()	9
3.1.3.7 setDamage()	9
3.1.3.8 setFacing()	9
3.1.3.9 setOffsets()	10
3.1.3.10 setOrigin()	10
3.1.3.11 setRotation()	10
3.1.3.12 setScale()	10
3.1.3.13 setThrustOffset()	11
3.1.4 Member Data Documentation	11
3.1.4.1 attachmentOffsetLeft	11
3.1.4.2 attachmentOffsetRight	11
3.1.4.3 attackType	11
3.1.4.4 collisionRect	11
3.1.4.5 damage	11
3.1.4.6 facing	11
3.1.4.7 frameCellHeight	12
3.1.4.8 frameCellWidth	12
3.1.4.9 origin	12
3.1.4.10 rotation	12
3.1.4.11 scale	12
3.1.4.12 sourceRect	12
3.1.4.13 spriteSheetGridSize	12
3.1.4.14 texture	12
3.1.4.15 thrustOffset	12

4 File Documentation	13
4.1 weapon.cpp File Reference	13
4.2 weapon.h File Reference	13
4.2.1 Detailed Description	13
4.2.2 Enumeration Type Documentation	14
4.2.2.1 AttackType	14
4.3 weapon.h	14

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Weapon	Represents a weapon sprite with rendering, rotation, and collision detection	5
------------------------	--	-------------------

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

weapon.cpp	13
weapon.h	
Weapon rendering and collision system for 2D sprite-based games	13

Chapter 3

Class Documentation

3.1 Weapon Class Reference

Represents a weapon sprite with rendering, rotation, and collision detection.

```
#include <weapon.h>
```

Public Member Functions

- [Weapon](#) (Texture2D *sharedTexture, Vector2 position, Vector2 sheetSize={10, 3}, float [scale](#)=1.0f)
Constructs a weapon from sprite sheet grid coordinates.
- [Weapon](#) (Texture2D *sharedTexture, int frameIndex, Vector2 sheetSize={10, 3}, float [scale](#)=1.0f)
Constructs a weapon from a flat frame index in the sprite sheet.
- [~Weapon](#) ()=default
- void [Draw](#) (Vector2 warriorPos, int [facing](#), bool showBounds=false)
Draws the weapon sprite relative to a character position.
- const void [setOffsets](#) (Vector2 rightOffset, Vector2 leftOffset)
Sets the attachment offset for each facing direction.
- Rectangle [computeCollisionRec](#) (Vector2 warriorPos, int [facing](#)) const
Computes collision rectangle without modifying internal state.
- const Rectangle & [getCollisionRect](#) () const
Gets the cached collision rectangle from last [Draw\(\)](#) call.
- const void [setRotation](#) (float newRotation)
Sets the weapon's rotation angle.
- const void [setScale](#) (float newScale)
Sets the weapon's render scale.
- const void [setOrigin](#) (Vector2 newOrigin)
Sets the weapon's origin point for rotation.
- const void [setFacing](#) (int f)
Sets the weapon's facing direction.
- const void [setAttackType](#) (AttackType type)
Sets the weapon's attack behavior pattern.
- const void [setThrustOffset](#) (float offset)
Sets the thrust offset for thrust-type attacks.
- const AttackType [getAttackType](#) () const
Gets the weapon's current attack type.
- const void [setDamage](#) (float dmg)
Sets the weapon's damage value.
- const float [getDamage](#) () const
Gets the weapon's damage value.

Private Attributes

- Texture2D * [texture](#) {nullptr}
Pointer to shared sprite sheet texture (not owned).
- Rectangle [sourceRect](#) {0, 0, 0, 0}
Source rectangle in sprite sheet (pixels).
- Vector2 [spriteSheetGridSize](#) {}
Grid dimensions of sprite sheet (columns, rows).
- float [rotation](#) {0.0f}
Current rotation angle in degrees.
- float [frameCellWidth](#) {0.0f}
Width of single sprite cell (pixels).
- float [frameCellHeight](#) {0.0f}
Height of single sprite cell (pixels).
- float [scale](#) {1.0f}
Render scale multiplier.
- Vector2 [origin](#) {0.f, 0.f}
Rotation origin in sprite-local coordinates (pixels, unscaled).
- Vector2 [attachmentOffsetRight](#) {35.f, 55.f}
Offset from character when facing right.
- Vector2 [attachmentOffsetLeft](#) {25.f, 55.f}
Offset from character when facing left.
- Rectangle [collisionRect](#) {0.f, 0.f, 0.f, 0.f}
Cached collision AABB from last [Draw\(\)](#) call.
- int [facing](#) {1}
Current facing direction (1 = right, -1 = left).
- [AttackType](#) [attackType](#) {AttackType::Swing}
Current attack behavior pattern.
- float [thrustOffset](#) {0.f}
Linear thrust distance for Thrust attacks (pixels).
- float [damage](#) {10.f}
Damage dealt by this weapon.

3.1.1 Detailed Description

Represents a weapon sprite with rendering, rotation, and collision detection.

The [Weapon](#) class handles sprite rendering from a shared texture atlas, supports multiple attack patterns, and provides collision detection with rotation support. Weapons do not own their texture and are designed for efficient reuse.

3.1.2 Constructor & Destructor Documentation

3.1.2.1 Weapon() [1/2]

```
Weapon::Weapon (
    Texture2D * sharedTexture,
    Vector2 position,
    Vector2 sheetSize = {10, 3},
    float scale = 1.0f) [inline]
```

Constructs a weapon from sprite sheet grid coordinates.

Parameters

<i>sharedTexture</i>	Pointer to the sprite sheet texture (not owned by weapon)
<i>position</i>	Grid position (column, row) in the sprite sheet
<i>sheetSize</i>	Grid dimensions of sprite sheet in cells (columns, rows)
<i>scale</i>	Render scale multiplier (1.0f = original size)

3.1.2.2 Weapon() [2/2]

```
Weapon::Weapon (
    Texture2D * sharedTexture,
    int frameIndex,
    Vector2 sheetSize = {10, 3},
    float scale = 1.0f) [inline]
```

Constructs a weapon from a flat frame index in the sprite sheet.

Parameters

<i>sharedTexture</i>	Pointer to the sprite sheet texture (not owned by weapon)
<i>frameIndex</i>	Zero-based frame index in row-major order
<i>sheetSize</i>	Grid dimensions of sprite sheet in cells (columns, rows)
<i>scale</i>	Render scale multiplier (1.0f = original size)

3.1.2.3 ~Weapon()

```
Weapon::~Weapon () [default]
```

3.1.3 Member Function Documentation

3.1.3.1 computeCollisionRec()

```
Rectangle Weapon::computeCollisionRec (
    Vector2 warriorPos,
    int facing) const
```

Computes collision rectangle without modifying internal state.

Parameters

<i>warriorPos</i>	Character's world position
<i>facing</i>	Direction character is facing (-1 = left, 1 = right)

Returns

Axis-aligned bounding box (AABB) for the rotated weapon sprite

3.1.3.2 Draw()

```
void Weapon::Draw (
    Vector2 warriorPos,
    int facing,
    bool showBounds = false)
```

Draws the weapon sprite relative to a character position.

Parameters

<i>warriorPos</i>	Character's world position (attachment point)
<i>facing</i>	Direction character is facing (-1 = left, 1 = right)
<i>showBounds</i>	If true, draws debug visualization (collision box, pivot, origin)

3.1.3.3 getAttackType()

```
const AttackType Weapon::getAttackType () const [inline]
```

Gets the weapon's current attack type.

Returns

Current attack behavior pattern

3.1.3.4 getCollisionRect()

```
const Rectangle & Weapon::getCollisionRect () const [inline]
```

Gets the cached collision rectangle from last [Draw\(\)](#) call.

Returns

Reference to the collision rectangle

3.1.3.5 `getDamage()`

```
const float Weapon::getDamage () const [inline]
```

Gets the weapon's damage value.

Returns

Current damage amount

3.1.3.6 `setAttackType()`

```
const void Weapon::setAttackType (  
    AttackType type) [inline]
```

Sets the weapon's attack behavior pattern.

Parameters

<i>type</i>	Attack type (Swing or Thrust)
-------------	-------------------------------

3.1.3.7 `setDamage()`

```
const void Weapon::setDamage (  
    float dmg) [inline]
```

Sets the weapon's damage value.

Parameters

<i>dmg</i>	Damage amount
------------	---------------

3.1.3.8 `setFacing()`

```
const void Weapon::setFacing (  
    int f) [inline]
```

Sets the weapon's facing direction.

Parameters

<i>f</i>	Facing direction (≥ 0 for right, < 0 for left)
----------	--

3.1.3.9 setOffsets()

```
const void Weapon::setOffsets (
    Vector2 rightOffset,
    Vector2 leftOffset) [inline]
```

Sets the attachment offset for each facing direction.

Parameters

<i>rightOffset</i>	Offset from character position when facing right
<i>leftOffset</i>	Offset from character position when facing left

3.1.3.10 setOrigin()

```
const void Weapon::setOrigin (
    Vector2 newOrigin) [inline]
```

Sets the weapon's origin point for rotation.

Parameters

<i>newOrigin</i>	Origin in sprite-local coordinates (pixels, unscaled)
------------------	---

3.1.3.11 setRotation()

```
const void Weapon::setRotation (
    float newRotation) [inline]
```

Sets the weapon's rotation angle.

Parameters

<i>newRotation</i>	Rotation in degrees (clockwise)
--------------------	---------------------------------

3.1.3.12 setScale()

```
const void Weapon::setScale (
    float newScale) [inline]
```

Sets the weapon's render scale.

Parameters

<i>newScale</i>	Scale multiplier (1.0f = original size)
-----------------	---

3.1.3.13 setThrustOffset()

```
const void Weapon::setThrustOffset (
    float offset) [inline]
```

Sets the thrust offset for thrust-type attacks.

Parameters

<i>offset</i>	Distance in pixels to thrust forward
---------------	--------------------------------------

3.1.4 Member Data Documentation

3.1.4.1 attachmentOffsetLeft

```
Vector2 Weapon::attachmentOffsetLeft {25.f, 55.f} [private]
```

Offset from character when facing left.

3.1.4.2 attachmentOffsetRight

```
Vector2 Weapon::attachmentOffsetRight {35.f, 55.f} [private]
```

Offset from character when facing right.

3.1.4.3 attackType

```
AttackType Weapon::attackType {AttackType::Swing} [private]
```

Current attack behavior pattern.

3.1.4.4 collisionRect

```
Rectangle Weapon::collisionRect {0.f, 0.f, 0.f, 0.f} [private]
```

Cached collision AABB from last [Draw\(\)](#) call.

3.1.4.5 damage

```
float Weapon::damage {10.f} [private]
```

Damage dealt by this weapon.

3.1.4.6 facing

```
int Weapon::facing {1} [private]
```

Current facing direction (1 = right, -1 = left).

3.1.4.7 frameCellHeight

```
float Weapon::frameCellHeight {0.0f} [private]
```

Height of single sprite cell (pixels).

3.1.4.8 frameCellWidth

```
float Weapon::frameCellWidth {0.0f} [private]
```

Width of single sprite cell (pixels).

3.1.4.9 origin

```
Vector2 Weapon::origin {0.f, 0.f} [private]
```

Rotation origin in sprite-local coordinates (pixels, unscaled).

3.1.4.10 rotation

```
float Weapon::rotation {0.0f} [private]
```

Current rotation angle in degrees.

3.1.4.11 scale

```
float Weapon::scale {1.0f} [private]
```

Render scale multiplier.

3.1.4.12 sourceRect

```
Rectangle Weapon::sourceRect {0, 0, 0, 0} [private]
```

Source rectangle in sprite sheet (pixels).

3.1.4.13 spriteSheetGridSize

```
Vector2 Weapon::spriteSheetGridSize {} [private]
```

Grid dimensions of sprite sheet (columns, rows).

3.1.4.14 texture

```
Texture2D* Weapon::texture {nullptr} [private]
```

Pointer to shared sprite sheet texture (not owned).

3.1.4.15 thrustOffset

```
float Weapon::thrustOffset {0.f} [private]
```

Linear thrust distance for Thrust attacks (pixels).

The documentation for this class was generated from the following files:

- [weapon.h](#)
- [weapon.cpp](#)

Chapter 4

File Documentation

4.1 weapon.cpp File Reference

```
#include "weapon.h"  
#include <cmath>  
#include <cmath>  
#include <cmath>
```

4.2 weapon.h File Reference

[Weapon](#) rendering and collision system for 2D sprite-based games.

```
#include "raylib.h"
```

Classes

- class [Weapon](#)
Represents a weapon sprite with rendering, rotation, and collision detection.

Enumerations

- enum class [AttackType](#) { [Swing](#) , [Thrust](#) }
Defines different weapon attack behavior patterns.

4.2.1 Detailed Description

[Weapon](#) rendering and collision system for 2D sprite-based games.

4.2.2 Enumeration Type Documentation

4.2.2.1 AttackType

enum class `AttackType` [strong]

Defines different weapon attack behavior patterns.

Enumerator

Swing	Rotate about origin point (swords, axes).
Thrust	Rotate then move linearly forward (spears, polearms).

4.3 weapon.h

[Go to the documentation of this file.](#)

```

00001
00005
00006 #include "raylib.h"
00007
00012 enum class AttackType
00013 {
00014     Swing,
00015     Thrust
00016 };
00017
00026 class Weapon
00027 {
00028 public:
00036     Weapon(Texture2D *sharedTexture, Vector2 position, Vector2 sheetSize = {10, 3}, float scale =
1.0f)
00037     : spriteSheetGridSize(sheetSize), scale(scale)
00038     {
00039         texture = sharedTexture;
00040
00041         // Calculate single-cell width/height
00042         frameCellWidth = static_cast<float>(texture->width) / spriteSheetGridSize.x;
00043         frameCellHeight = static_cast<float>(texture->height) / spriteSheetGridSize.y;
00044
00045         // Use column * cellWidth and row * cellHeight
00046         sourceRect = {static_cast<int>(position.x) * frameCellWidth,
00047             static_cast<int>(position.y) * frameCellHeight,
00048             frameCellWidth, frameCellHeight};
00049
00050         origin = {0.f, frameCellHeight}; // bottom-left by default (x=0, y=height)
00051     }
00052
00060     Weapon(Texture2D *sharedTexture, int frameIndex, Vector2 sheetSize = {10, 3}, float scale = 1.0f)
00061     : spriteSheetGridSize(sheetSize), scale(scale)
00062     {
00063         texture = sharedTexture;
00064
00065         frameCellWidth = static_cast<float>(texture->width) / spriteSheetGridSize.x;
00066         frameCellHeight = static_cast<float>(texture->height) / spriteSheetGridSize.y;
00067
00068         int cols = static_cast<int>(spriteSheetGridSize.x);
00069         int col = frameIndex % cols;
00070         int row = frameIndex / cols;
00071
00072         sourceRect = {col * frameCellWidth, row * frameCellHeight, frameCellWidth, frameCellHeight};
00073         origin = {0.f, frameCellHeight};
00074     }
00075
00076     ~Weapon() = default;
00077
00084     void Draw(Vector2 warriorPos, int facing, bool showBounds = false);
00085
00091     const void setOffsets(Vector2 rightOffset, Vector2 leftOffset)
00092     {
00093         attachmentOffsetRight = rightOffset;
00094         attachmentOffsetLeft = leftOffset;
00095     }
00096
00103     Rectangle computeCollisionRec(Vector2 warriorPos, int facing) const;

```

```
00104
00109     const Rectangle &getCollisionRect() const { return collisionRect; }
00110
00115     const void setRotation(float newRotation) { rotation = newRotation; }
00116
00121     const void setScale(float newScale) { scale = newScale; }
00122
00127     const void setOrigin(Vector2 newOrigin) { origin = newOrigin; }
00128
00133     const void setFacing(int f) { facing = (f >= 0 ? 1 : -1); }
00134
00139     const void setAttackType(AttackType type) { attackType = type; }
00140
00145     const void setThrustOffset(float offset) { thrustOffset = offset; }
00146
00151     const AttackType getAttackType() const { return attackType; }
00152
00157     const void setDamage(float dmg) { damage = dmg; }
00158
00163     const float getDamage() const { return damage; }
00164
00165 private:
00166     Texture2D *texture{nullptr};
00167     Rectangle sourceRect{0, 0, 0, 0};
00168     Vector2 spriteSheetGridSize{};
00169     float rotation{0.0f};
00170     float frameCellWidth{0.0f};
00171     float frameCellHeight{0.0f};
00172     float scale{1.0f};
00173     Vector2 origin{0.f, 0.f};
00174     Vector2 attachmentOffsetRight{35.f, 55.f};
00175     Vector2 attachmentOffsetLeft{25.f, 55.f};
00176     Rectangle collisionRect{0.f, 0.f, 0.f, 0.f};
00177     int facing{1};
00178
00179     AttackType attackType{AttackType::Swing};
00180     float thrustOffset{0.f};
00181     float damage{10.f};
00182 };
```

